

System Software Development 2 – SSD226C 2026 Notes

Introduction to Django

Django is a high-level Python web framework that simplifies web development by offering built-in tools for common tasks like user authentication, database management, URL routing, and security. Unlike microframeworks like Flask, which require many third-party packages, Django provides a comprehensive, secure, and scalable solution out of the box. It follows Python's "batteries-included" philosophy, allowing developers to focus on building unique features instead of reinventing the wheel. With a strong community, regular updates, and a proven track record, Django is a powerful choice for building modern web applications.

WHY DJANGO?

BUILD SMARTER, NOT HARDER



Built-In Tools

- User authentication
- Admin interface
- URL routing



Secure & Scalable

- Regular security fixes
- Performance tuning



Community & Support

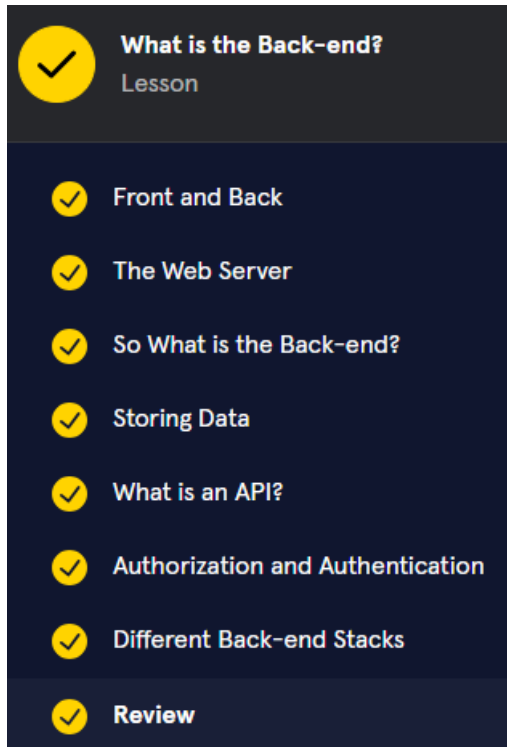
- Active development
- Rich documentation

„Django lets you focus on what makes your app unique — not on reinventing the basics.“

 **Powered by Python. Trusted by developers worldwide.**

SOME IMPORTANT BACKGROUND => WHAT IS THE BACK-END

Before we dive into the different aspects of the Django framework some background information regarding the back end of web applications is needed. In this unit the following topics will be investigated:



Front and Back

In this lesson, we'll explain what makes up the [back-end](#) of a web application or website. The back-end can feel very abstract, but it becomes clearer when we explain it in terms of the front-end! To oversimplify a bit, the [front-end](#) is the parts of a webpage that a visitor can interact with and see.

Various tools and frameworks can be used to make a webpage, but, at its core, the front-end is composed of JavaScript, CSS, HTML, and other *static assets*, such as images or videos. Static assets are files that don't change. When a visitor navigates to a webpage, these assets are sent to their browser.

Visiting a simple website is like ordering delivery from a restaurant: we place an order for our meal, and, once it's delivered to us, we have it entirely in our possession. In this analogy, we can think of the front-end as everything that's dropped off with the delivery: the containers, the utensils, and the food itself.

You'll sometimes hear front-end development referred to as *client-side* development. Our instinct might be to think of the client as the human visitor or user of a website, but when referring to the client in web development, we're usually referring to the non-human requester of content. In the case of visiting a website, the client is the browser, but in other circumstances, a client might be another application, a mobile device, or even a "smart" appliance!

While the front-end is the part of the website that makes it to the browser, the back-end consists of all the behind-the-scenes processes and data that make a website function and send resources to clients.

Instructions

Watch the video linked below to learn about the differences between the front-end and back-end.

<https://www.youtube.com/watch?v=YNVjL-3SMIc>

The Web Server

We talked about how the front-end consists of the information sent to a client so that a user can see and interact with a website, but where does the information come from? The answer is a web server.

The word “server” can mean a lot of things in computing, but we’re going to focus on web servers specifically. A web server is a process running on a computer that listens for incoming requests for information over the internet and sends back responses. Each time a user navigates to a website on their browser, the browser makes a request to the web server of that website. Every website has at least one web server. A large company like Facebook has thousands of powerful computers running web servers in facilities located all around the world which are listening for requests, but we could also run a simple web server from our own computer!

The specific format of a request (and the resulting response) is called the Protocol. You might be familiar with the protocol used to access websites: HTTP. When a visitor navigates to a website on their browser, similarly to how one places an order for takeout, they make an HTTP request for the resources that make up that site.

For the simplest websites, a client makes a single request. The web server receives that request and sends the client a response containing everything needed to view the website. This is called a static website. This doesn’t mean the website is not interactive. As with the individual static assets, a website is static because once those files are received, they don’t change or move. A static website might be a good choice for a simple personal website with a short bio and family photos. A user navigating Twitter, however, wants access to new content as it’s created, which a static website couldn’t provide.

A static website is like ordering takeout, but modern web applications are like dining in person at a sit-down restaurant. A restaurant patron might order drinks, different courses, make substitutions, or ask questions of the waiter. To accomplish this level of complexity, an equally complex back-end is required.

So What is the Back-end?

When a user navigates to google.com, their request specifies the [URL](#) but not the filename for today’s [Google Doodle](#). A Google Doodle is a special, temporary alteration of the logo on Google’s homepages intended to commemorate holidays,

events, achievements, and historical figures. The web application's [back-end](#) will need to hold the logic for deciding which assets to send. Moreover, modern web applications often cater to the specific user rather than sending the same files to every visitor of a webpage. This is known as *dynamic content*.

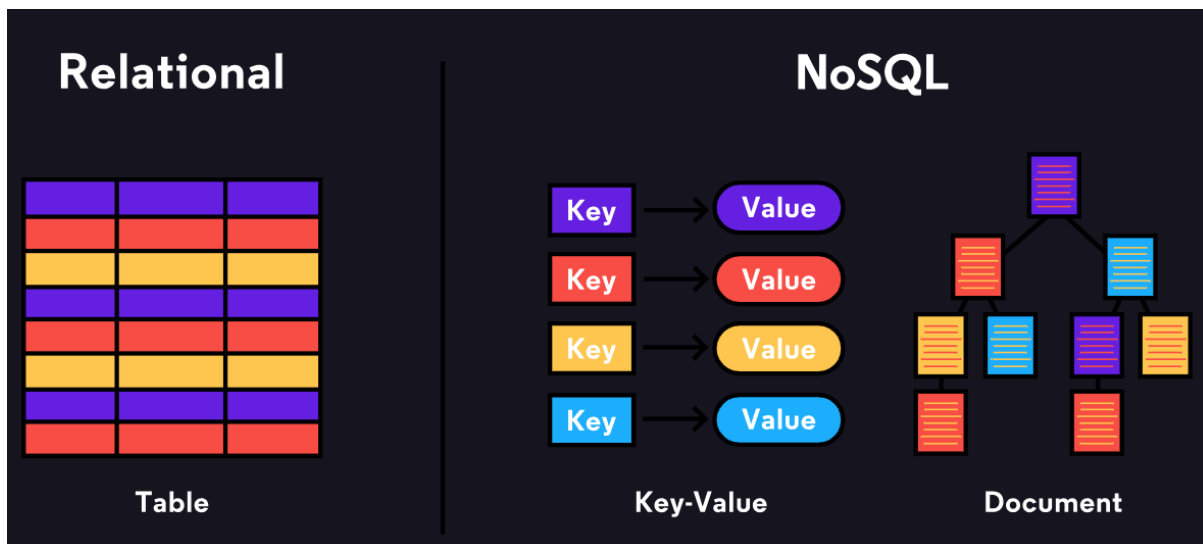
When we eat at a restaurant, we'll order to our tastes, make substitutions, etc; the result is a dining experience unique to us. Aside from that, there's a lot happening behind the scenes to make a restaurant work: ingredients are ordered from suppliers, new menus are designed, and employees' schedules are created. Similarly, to make a web application that runs smoothly, the back-end is doing a lot more than simply sending assets to browsers.

The collection of programming logic required to deliver dynamic content to a client, manage security, process payments, and myriad other tasks is sometimes known as the "application" or *application server*. The application can be responsible for anything from sending an email confirmation after a purchase to running the complicated algorithms a search engine uses to give us meaningful results.

Storing Data

You've probably heard that data is a big deal. By some measures, 90% of the world's data has been generated in just the past two years! From a stored credit card number on an e-commerce site to the timestamp when you hit pause on Netflix, modern web applications collect a lot of data. For that data to be useful, it has to be organized and stored somewhere. The back-ends of modern web applications include some sort of database, often more than one. Databases are collections of information. There are many different databases, but we can divide them into two types: relational databases and non-relational databases (also known as NoSQL databases). Whereas relational databases store information in tables with columns and rows, non-relational databases might use other systems such as key-value pairs or a document storage model. SQL, Structured Query Language, is a programming language for accessing and changing data stored in relational databases. Popular relational databases include MySQL and PostgreSQL while popular NoSQL databases include MongoDB and Redis.

In addition to the database itself, the back-end needs a way to programmatically access, change, and analyze the data stored there.



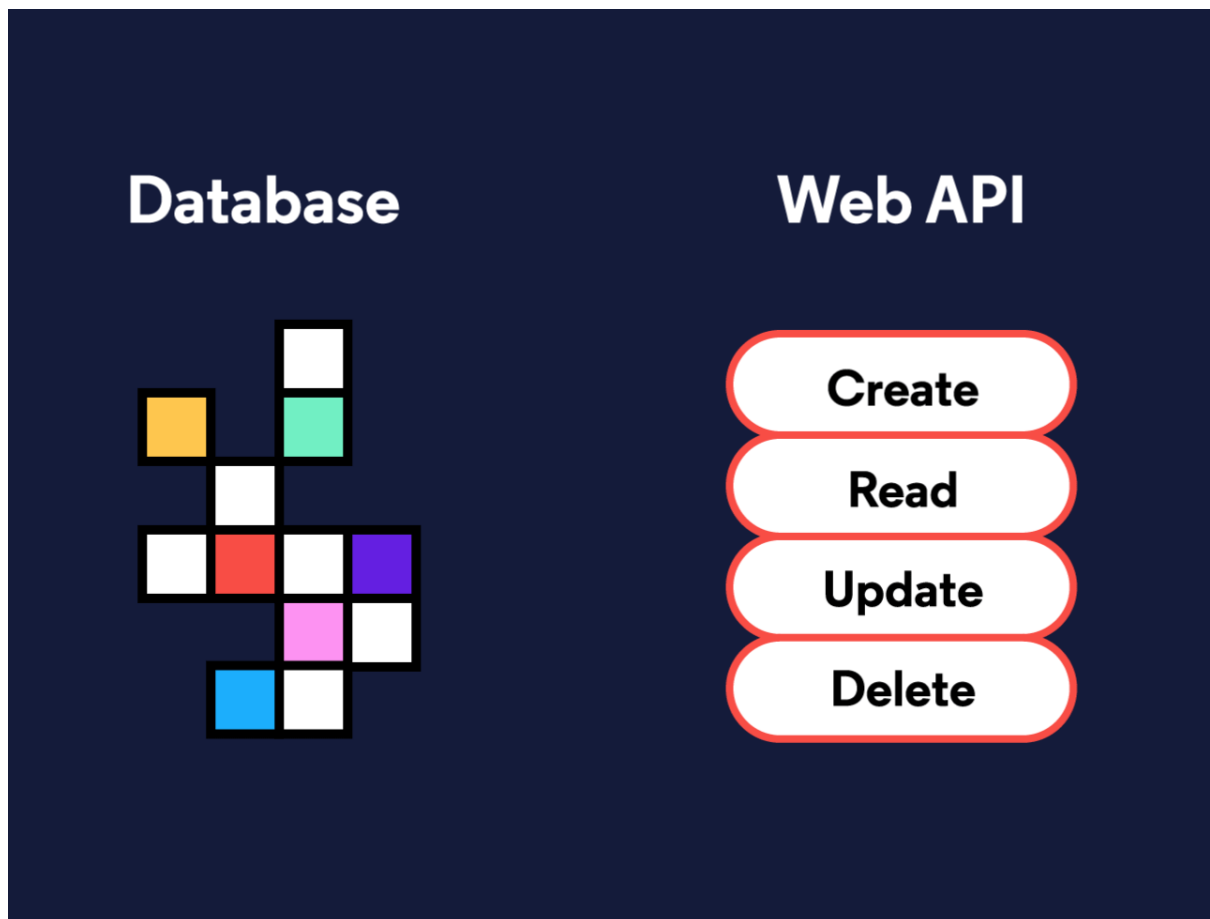
What is an API?

When a user navigates to a specific item for sale on an e-commerce site, the price listed for that item is stored in a [database](#), and when they purchase it, the database will need to be updated with the correct inventory for that item type. In fact, much of what the [back-end](#) entails is reading, updating, or deleting information stored in a database.

In order to have consistent ways of interacting with data, a back-end will often include a *web API*. [API](#) stands for **A**pplication **P**rogramming **I**nterface and can mean a lot of different things, but a *web API* is a collection of predefined ways of, or rules for, interacting with a web application's data, often through an [HTTP](#) request-response cycle. Unlike the HTTP requests a client makes when a user navigates to a website's [URL](#), this type of request indicates how it would like to interact with a web application's data (create new data, read existing data, update existing data, or delete existing data), and it receives some data back as a response.

Let's walk through the example of making an online purchase again, but this time, we'll imagine how the application's web API might be used. When a user presses the button to submit an order, that will trigger a request to send them to a different view of the website, an order confirmation page, but an additional request will be triggered from the [front-end](#), unseen by the user, to the web API so that the database can be updated with the information from the order.

Some web APIs are open to the public. [Instagram](#), for example, has an API that other developers can use to access some of the data Instagram stores. Others are only used by the web application internally.



Authorization and Authentication

Two other concepts we'll want our server-side logic to handle are *authentication* and *authorization*.

Authentication is the process of validating the identity of a user. One technique for authentication is to use logins with usernames and passwords. These credentials need to be securely stored in the [back-end](#) on a [database](#) and checked upon each visit. Web applications can also use external resources for authentication. You've likely logged into a website or application using your Facebook, Google, or Github credentials; that's also an authentication process.

Authorization controls which users have access to which resources and actions. Certain application views, like the page to edit a social media personal profile, are only accessible to that user. Other activities, like deleting a post, are often similarly restricted.

When building a robust web application back-end, we need to incorporate both authentication (Who is this user? Are they who they claim to be?) and authorization (Who is allowed to do and see what?) into our server-side logic to make sure we're creating secure, personalized, and dynamic content.

Different Back-end Stacks

Unlike the front-end, which must be built using HTML, CSS, and JavaScript, there's a lot of flexibility in which technologies can be used in order to create the back-end of a web application. Developers can construct back-ends in many different languages like PHP, Java, JavaScript, Python, and more.

You don't need to reinvent the wheel to create a robust back-end. Instead, most developers make use of *frameworks* which are collections of tools that shape the organization of your back-end and provide efficient ways of accomplishing otherwise difficult tasks.

There are numerous [back-end frameworks](#) from which developers can choose. Here are a few examples:

Framework	Language
Laravel	PHP
Express.js	JavaScript (runs in the Node environment)
Ruby on Rails	Ruby
Spring	Java
JSE	Java
Flask	Python
Django	Python
ASP.NET	C#

The collection of technologies used to create the front-end and back-end of a web application is referred to as a *stack*. This is where the term *full-stack developer* comes from; rather than working in either the front-end or the back-end exclusively, a full-stack developer works in both.

For example, [the MEAN stack](#) is a technology stack for building web applications that uses MongoDB, Express.js, AngularJS, and Node.js: MongoDB is used as the database, Node.js with Express.js for the rest of the back-end, and Angular is used as a front-end framework. While the [LAMP Stack](#), sometimes considered the archetypal stack, uses Linux, Apache, MySQL, and PHP.

Review

In order to deliver the [front-end](#) of a website or web application to a user, a lot needs to happen behind the scenes on the back-end! Understanding what makes up the [back-end](#) can be overwhelming because the back-end has a lot of different parts, and different websites or web applications can have dramatically different back-ends. We covered a lot in this lesson, so let's review what we learned:

- The front-end of a website or application consists of the HTML, CSS, JavaScript, and static assets sent to a client, like a web browser.

- A web [server](#) is a process running on a computer somewhere that listens for incoming requests for information over the internet and sends back responses.
- Storing, accessing, and manipulating data is a large part of a web application's back-end
- Data is stored in databases which can be relational databases or NoSQL databases.
- The server-side of a web application, sometimes called the application server, handles important tasks such as authorization and authentication.
- The back-end of web application often has a web [API](#) which is a way of interacting with an application's data through [HTTP](#) requests and responses.
- Together the technologies used to build the front-end and back-end of a web application are known as the stack, and many different languages and frameworks can be used to build a robust back-end.

Now that you have a sense for server-side web development and what the back-end is, you're ready to dive in and learn about the different parts in more depth!

